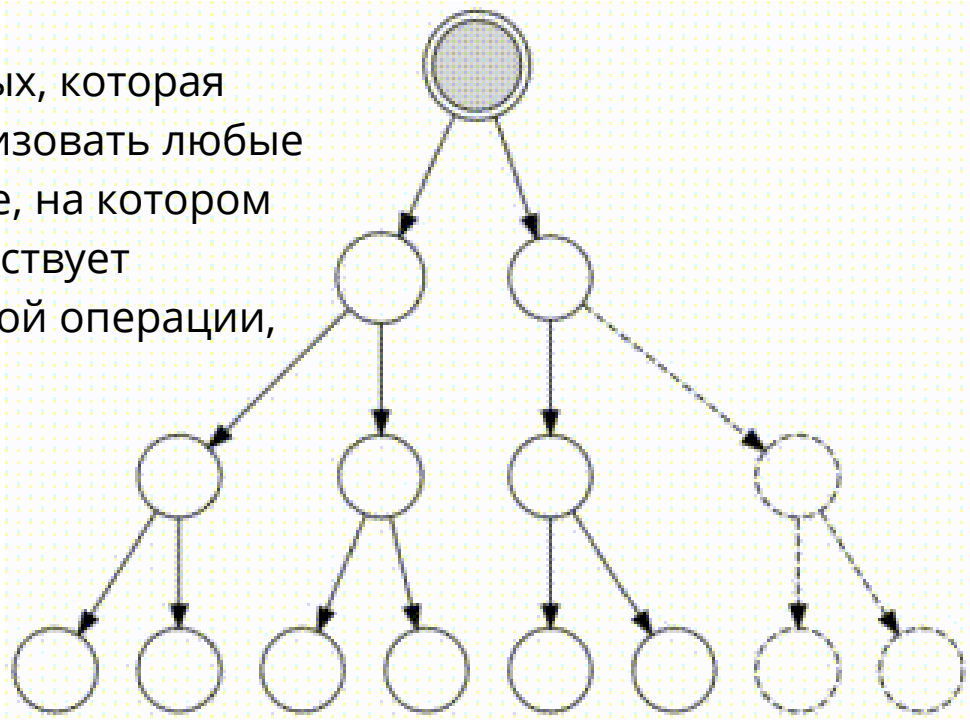
АиСД

Красно-черные деревья
Дерево отрезков
Декартово дерево

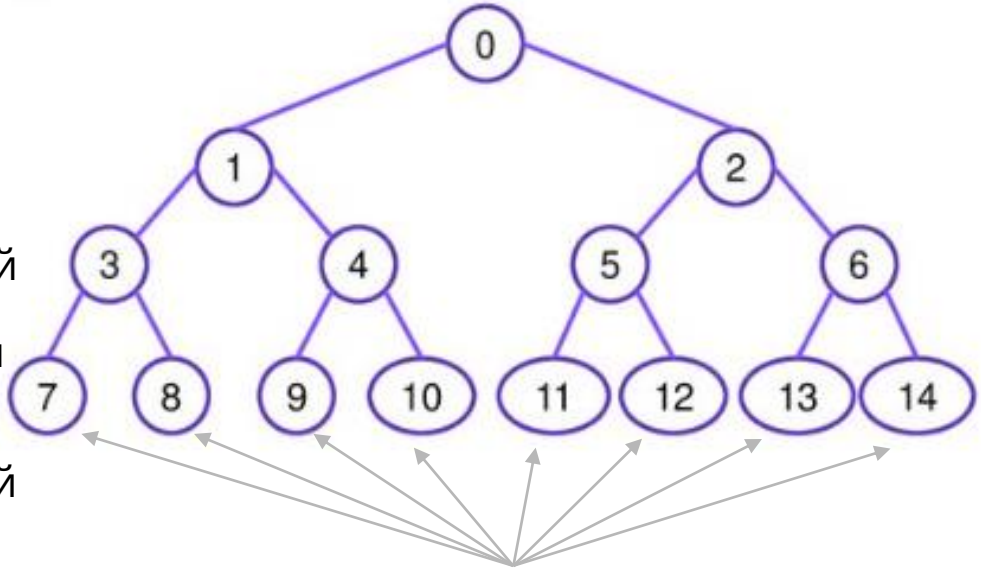
Дерево отрезков

Дерево отрезков - это структура данных, которая позволяет за асимптотику O(logn) реализовать любые операции, определяемые на множестве, на котором данная операция ассоциативна, и существует нейтральный элемент относительно этой операции, то есть на моноиде.



Описание структуры

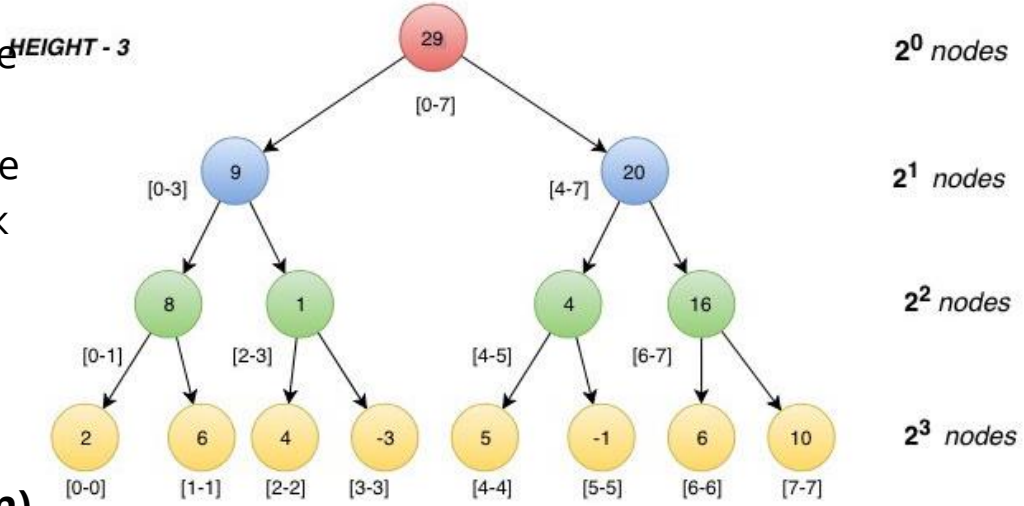
- Дерево отрезков - корневое дерево
- Листья - элементы исходного массива
- Другие вершины - имеют два ребенка
- Каждой вершине в соответствие поставлен интервал, являющийся объединением интервалов ее детей (если у вершины есть дети), либо интервал, содержащий конкретный элемент массива (для листьев).
- В каждой вершины хранится значение некоторой ассоциативной функции **f** на данном интервале



Исходный массив
[7, 8, 9, 10, 11, 12, 13, 14]

Построение дерева отрезков

- Сначала запишем значения элементов a[i] в соответствующие листья дерева.
- Рекурсивно подсчитаем значение вершин предыдущего уровня как сумму значений в двух листьях.



Асимптотика построения дерева отрезков составит, таким образом, **O(n)**.

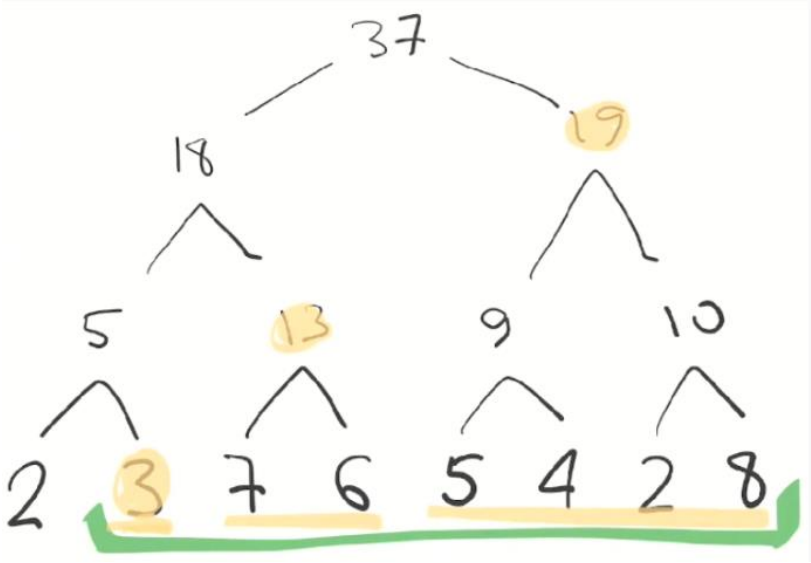
Корень этого дерева соответствует отрезку [0, n)

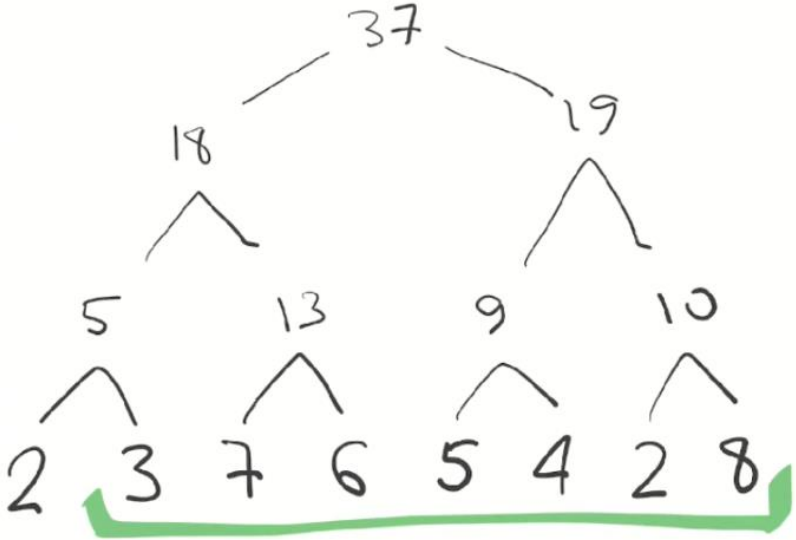
Свойства:

- Высота дерева отрезков равна Θ(logn): на каждом новом уровне длина отрезка уменьшается вдвое.
- Любой полуинтервал разбивается на O(logn) неперекрывающихся полуинтервалов, соответствующих вершинам дерева.
- Дерево содержит менее 2n вершин.
- При n, отличных от степеней двойки, не все уровни дерева отрезков будут полностью заполнены.

Запрос суммы отрезка

- Чтобы найти сумму отрезка, нужно сложить элементы, которые, так или иначе, покрывают наш отрезок:





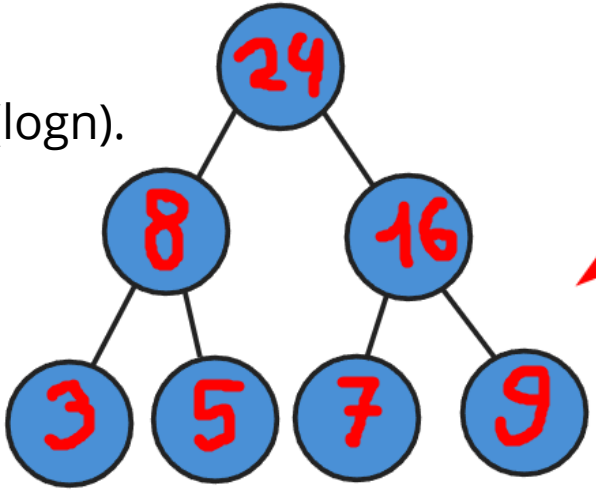
Результат: 3 + 13 + 19 = 35.

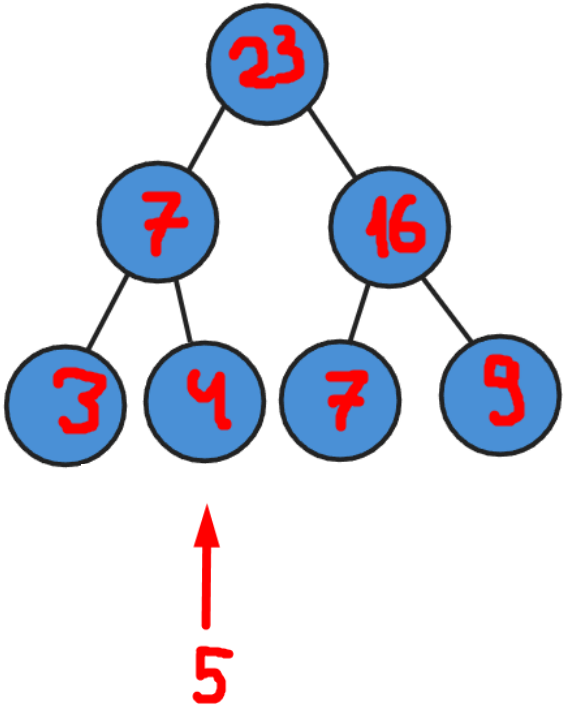
Если бы отрезок состоял из тысячи элементов, достаточно было бы сложить в среднем 10 элементов!

Запрос обновления элемента

- Изменить значение в листе, а потом заново пересчитать все значения в вершинах на пути до корня.

Время работы O(logn).





Применение

- С помощью дерева отрезков можно искать сумму чисел, максимум или минимум, но и любую функцию, удовлетворяющую свойству ассоциативности.
- Выполнения массовых операций на многомерных структурах данных.
- Поиск НОД / НОК.
- Поиск префикса массива с заданной суммой.
- Персистентные структуры данных.

Декартово дерево

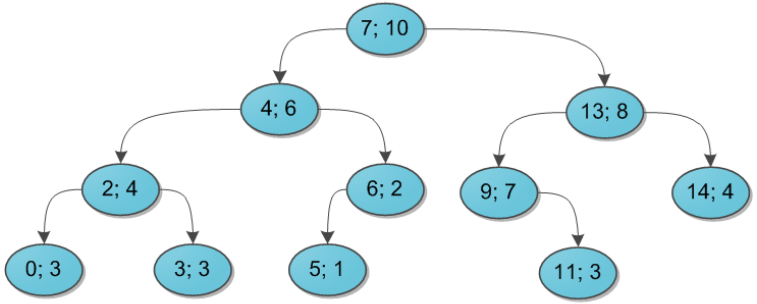
Рене Декарт не является создателем декартова дерева, но он является создателем декартовой системы координат, которую мы все знаем и любим.

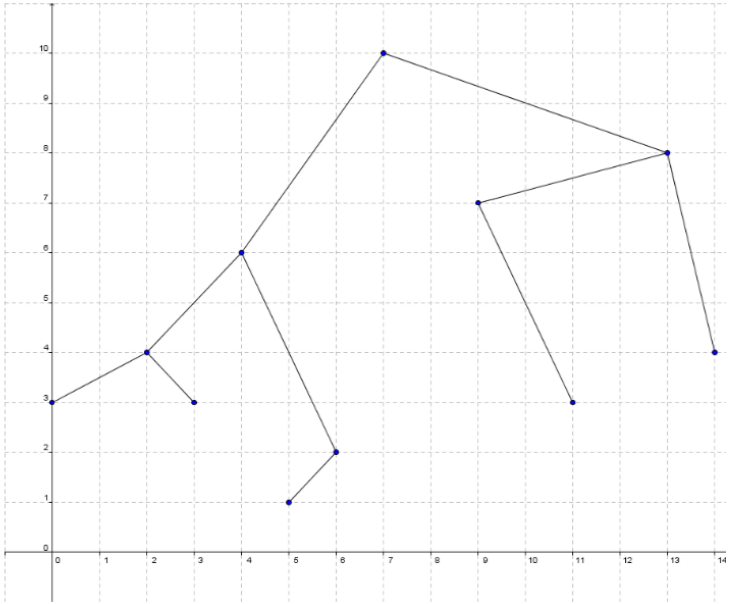
Декартово дерево же определяется и строится так:

- Нанесём на плоскость набор из точек. И **x** зачем-то назовем ***ключом***, а **y** ***приоритетом***.
- Выберем **самую верхнюю точку** *(с наибольшим **y**, а если таких несколько — любую)* и назовём её **корнем**.
- От всех вершин, лежащих слева *(с меньшим **x**)* от корня, рекурсивно запустим этот же процесс. Если слева была хоть одна вершина, то присоединим корень левой части в качестве левого сына текущего корня.
- Аналогично, запустимся от правой части и добавим корню правого сына.

Декартово дерево

Если нарисовать получившуюся структуру на плоскости, то получится действительно дерево — по традиции, корнем вверх:

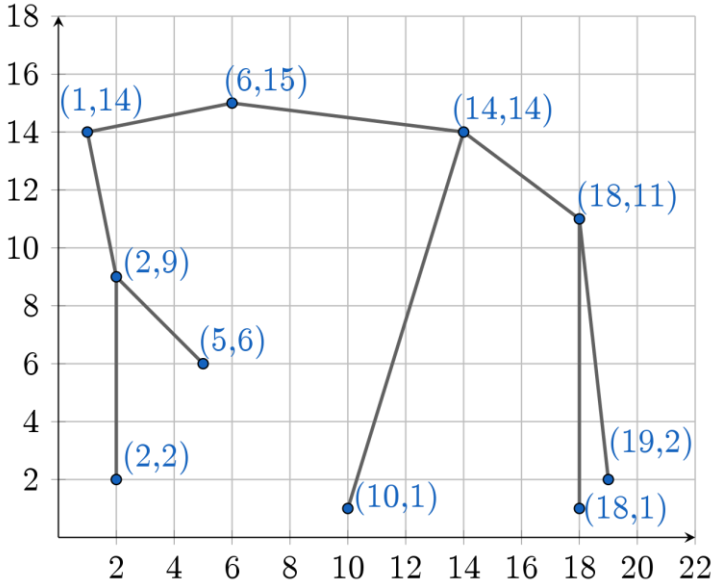




Заметим, что если все y и x различны, то дерево строится однозначно.

Декартово дерево

Таким образом, декартово дерево — это одновременно бинарное дерево по x и куча по y.



В декартовом дереве логарифмическая высота дерева гарантируется не инвариантами и эвристиками, а законами теории вероятностей:

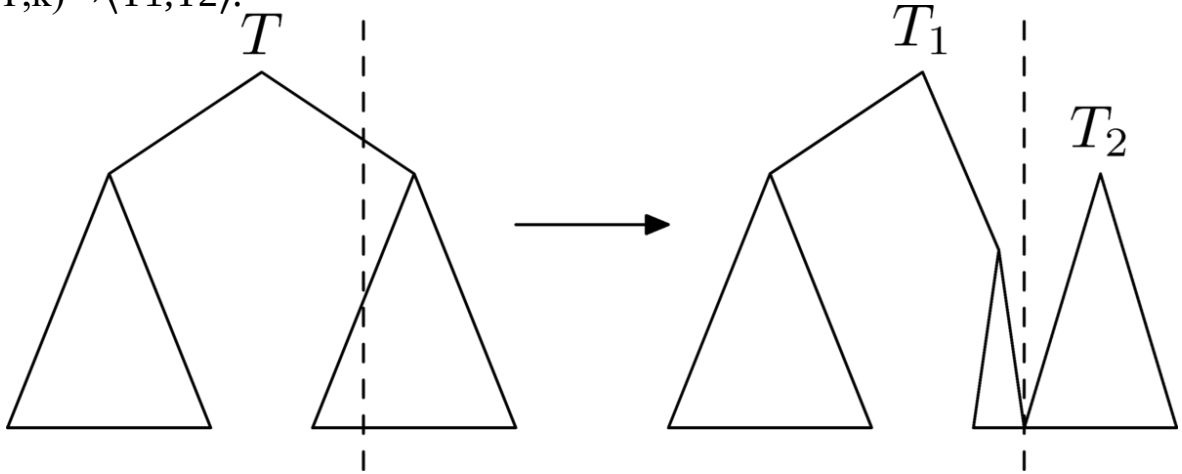
- оказывается, что если все приоритеты (y) выбирать случайно, то средняя глубина вершины будет логарифмической.

Поэтому ДД ещё называют **рандомизированным деревом поиска**.

Теорема. Ожидание глубины вершины в декартовом дереве равно $O(\log n)$.

Операции в декартовом дереве

Операция **split** (разрезать) позволяет сделать следующее: разрезать исходное дерево T по ключу k. Возвращать она будет такую пару деревьев ⟨T1,T2⟩, что в дереве T1 ключи меньше k, а в дереве T2 все остальные: split(T,k)→⟨T1,T2⟩.

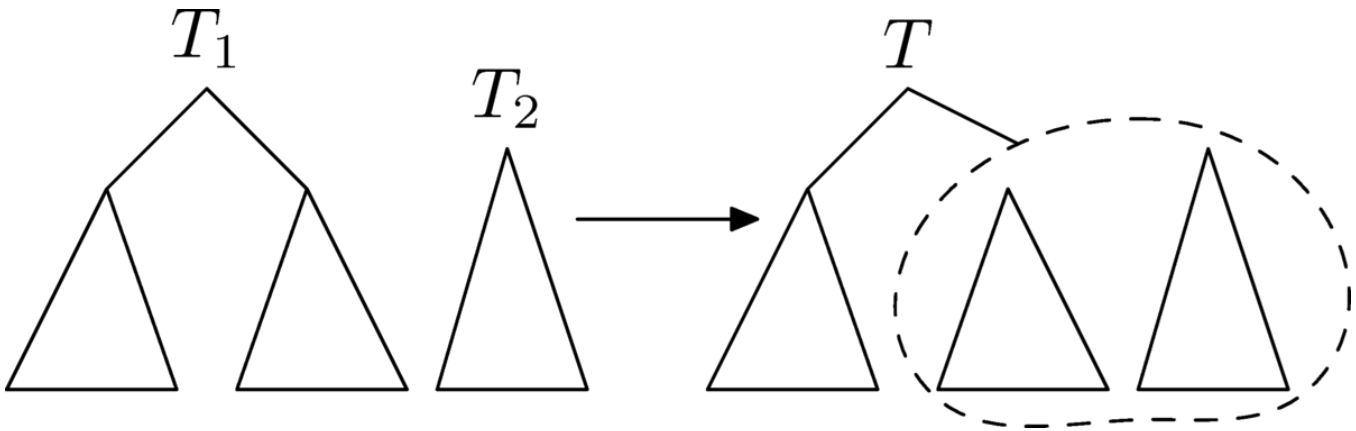


Оценим время работы операции **split**. Во время выполнения вызывается одна операция split для дерева хотя бы на один меньшей высоты и делается еще **O(1)** операций. Тогда итоговая трудоемкость этой операции равна **O(h)**, где **h** — высота дерева.

Операции в декартовом дереве

merge

С помощью этой операции можно слить два декартовых дерева в одно. Все ключи в первом(левом) дереве должны быть меньше, чем ключи во втором(правом). В результате получается дерево, в котором есть все ключи из первого и второго деревьев: merge(T1,T2)→{T}



Операции в декартовом дереве

merge

Пусть нужно слить деревья **T1** и **T2**:
Тогда, очевидно, у результирующего дерева **T** есть корень.

1. Корнем станет вершина из T1 или T2 с наибольшим приоритетом **y**.
    a. Но вершина с самым большим **y** из всех вершин деревьев T1 и T2 может быть только либо корнем T1, либо корнем T2.
    b. Если **y** корня T1 больше **y** корня T2, то он и будет являться корнем.
        i. Тогда левое поддерево T совпадет с левым поддеревом T1.
            1. Справа же нужно подвесить объединение правого поддерева T1 и дерева T2.

Случай, в котором корень T2 имеет больший y, чем корень T1, симметричен этому.

Рассуждая аналогично операции split, приходим к выводу, что трудоёмкость операции **merge** равна **O(h)**, где **h** — высота дерева.

Операции в декартовом дереве

merge и **split** сами по себе не очень полезные, но помогут написать все остальное.

Вот так, например, будет выглядеть код, добавляющий в дерево.

```
Node *root = 0;

void insert (int x) {
    Pair q = split(root, x);
    Node *t = new Node(x);
    root = merge(q.first, merge(t,
q.second));
}
```

Область применимости

Декартово дерево не является самобалансирующимся в обычном смысле, и применяют его по следующим причинам:

- Проще реализуется по сравнению, например, с настоящими самобалансирующимися деревьями вроде красно-чёрного.
- Хорошо ведёт себя «в среднем», если ключи y раздать случайно.
- Типичная для сортирующего дерева операция «разделить по ключу x на „меньше x_0“ и „не меньше x_0“» работает за $O(h)$, где h — высота дерева. На красно-чёрных деревьях придётся восстанавливать балансировку и окраску узлов.

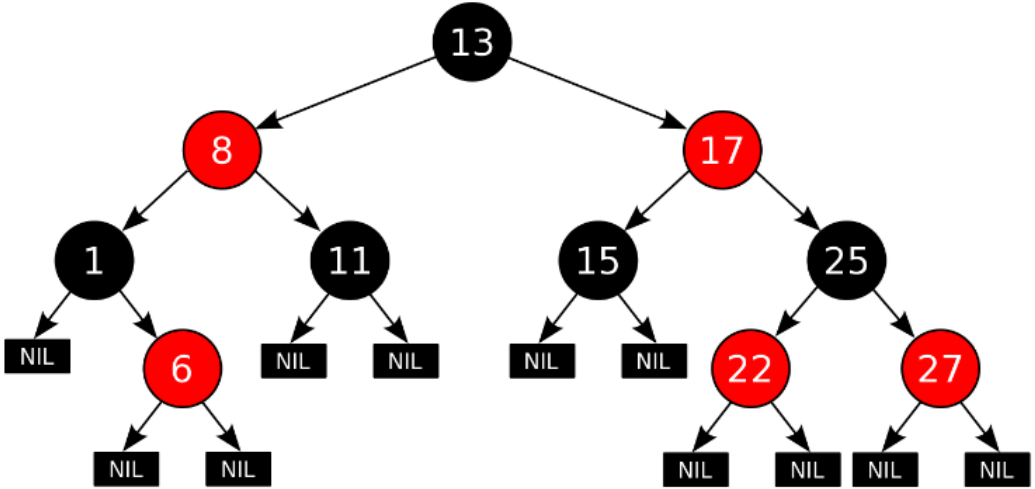
Недостатки декартового дерева:

- Большие накладные расходы на хранение: вместе с каждым элементом хранятся два-три указателя и случайный ключ y.
- Скорость доступа $O(n)$ в худшем, хотя и маловероятном, случае. Поэтому декартово дерево недопустимо, например, в ядрах ОС.

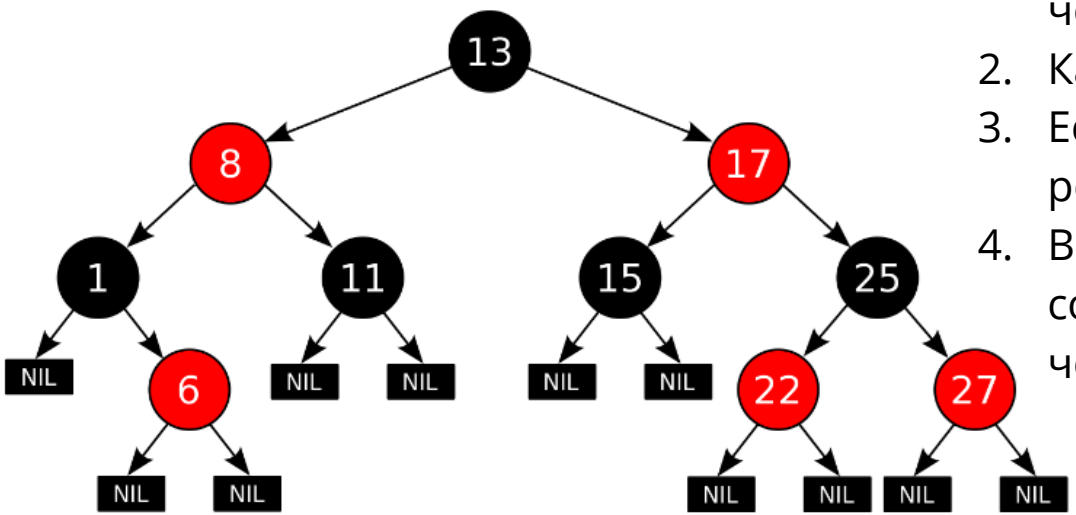
Красно-чёрное дерево

Красно-чёрное дерево — двоичное дерево поиска, в котором баланс осуществляется на основе "цвета" узла дерева.

Узел принимает только два значения: "красный" и "чёрный".



Принципы организации:



1. Каждая вершина - либо красная, либо черная
2. Каждый лист - черный
3. Если вершина красная, оба ее ребенка черные
4. Все пути, идущие от корня к листьям, содержат одинаковое количество черных вершин

Корень дерева должен быть черным!

Применение

- Ассоциативные массивы (Так, например, в стандартной библиотеке STL языка C++ контейнер map реализован на основе красно-чёрного дерева).
- Приложения реального времени.
- Вычислительная геометрия.
- Completely Fair Scheduler используется в текущих ядрах Linux.
- Реализация системного вызова epoll

Методы сжатия архиваторов. LZ

Словарный метод (LZ)

Наиболее распространенный метод. Используется словарь, состоящий из последовательностей данных или слов. При сжатии эти слова заменяются на их коды из словаря. В наиболее распространенном варианте реализации в качестве словаря выступает сам исходный блок данных.

ВХОДНАЯ ФРАЗА (В СЛОВАРЬ)	КОД	ПОЗИЦИЯ СЛОВАРЯ
		0
"К"	<0,'К'>	1
"Р"	<0,'Р'>	2
"А"	<0,'А'>	3
"С"	<0,'С'>	4
"Н"	<0,'Н'>	5
"АЯ"	<3,'Я'>	6
	<0,' '>	7
"КР"	<1,'Р'>	8
"АС"	<3,'С'>	9
"КА"	<1,'А'>	10

Методы сжатия архиваторов. LZ

Основным параметром словарного метода является размер словаря. Чем больше словарь, тем больше эффективность. Однако для неоднородных данных чрезмерно большой размер может быть вреден, так как при резком изменении типа данных словарь будет заполнен неактуальными словами.

Кодируемая последовательность	Код	Последовательность в словарь	Код
К	00001	КУ	01011
У	00010	УК	01100
КУ	01011	КУШ	01101
Ш	00011	ШК	01110
К	00001	КА	01111
А	00100	АК	10000
КУ	01011	КУК	10001
КУШ	01101	КУШО	10010
О	00101	ОН	10011
Н	00110	НК	10100
КУК	10001	КУКУ	10101
У	00010	УП	10110
П	00111	ПИ	10111
И	01000	ИЛ	11000
Л	01001	ЛА	11001
АК	10000	АКА	11010
А	00100	АП	11011
П	00111	ПЮ	11100
Ю	01010	ЮШ	11101
Ш	00011	ШО	11110
ОН	10011		

Методы сжатия архиваторов. LZ

Для эффективной работы данного метода при сжатии требуется дополнительная память. Приблизительно на порядок больше, чем нужно для исходных данных словаря. Существенным преимуществом словарного метода является простая и быстрая процедура распаковки. Дополнительная память при этом не требуется. Такая особенность особенно важна, если необходим оперативный доступ к данным.

ВХОДНОЙ КОД	ПЕЧАТЬ	СЛОВАРЬ	ПОЗИЦИЯ СЛОВАРЯ
		ASCII+	0-255
0’К’	"К"	"КР"	256
0’Р’	"Р"	"РА"	257
0’А’	"А"	"АС"	258
0’С’	"С"	"СН"	259
0’Н’	"Н"	"НА"	260
0’А’	"А"	"АЯ"	261
0’Я’	"Я"	"Я "	262
0’ ’		"К"	263
<256>	"КР"	"КРА"	264
<258>	"АС"	"АСК"	265
0’К’	"К"	"КА"	266
0’А’	"А"		

Методы сжатия архиваторов. LZ

Идеи:

- кодировать не отдельные символы, а блоки
- последовательностям символов присваиваются числовые коды
- новая цепочка ⇒ занесение в словарь с новым кодом

- словарь строится по мере получения данных
- не нужны частоты символов ⇒ за один проход!

Методы сжатия архиваторов. LZ

Кодирование

- Начало.
- **Шаг 1.** Все возможные символы заносятся в словарь. Во входную фразу X заносится первый символ сообщения.
- **Шаг 2.** Считать очередной символ Y из сообщения.
- **Шаг 3.** Если Y — это символ конца сообщения, то выдать код для X, иначе:
  - Если фраза XY уже имеется в словаре, то присвоить входной фразе значение XY и перейти к **Шагу 2** ,
  - Иначе выдать код для входной фразы X, добавить XY в словарь и присвоить входной фразе значение Y. Перейти к **Шагу 2.**
- Конец.

Методы сжатия архиваторов. LZ

Декодирование

- Начало.
- **Шаг 1.** Все возможные символы заносятся в словарь. Во входную фразу X заносится первый код декодируемого сообщения.
- **Шаг 2.** Считать очередной код Y из сообщения.
- **Шаг 3.** Если Y — это конец сообщения, то выдать символ, соответствующий коду X, иначе:
  - Если фразы под кодом XY нет в словаре, вывести фразу, соответствующую коду X, а фразу с кодом XY занести в словарь.
  - Иначе присвоить входной фразе код XY и перейти к **Шагу 2** .
- Конец.

LZ78

Изначально словарь пуст, а алгоритм пытается закодировать первый символ.
На каждой итерации мы пытаемся увеличить кодируемый префикс, пока такой префикс есть в словаре.
Кодовые слова такого алгоритма будут состоять из двух частей — номера в словаре самого длинного найденного префикса **(pos)** и символа, который идет за этим префиксом **(next)**. При этом после кодирования такой пары префикс с приписанным символом добавляется в словарь, а алгоритм продолжает кодирование со следующего символа.

LZ78

Пример

В этот раз для примера возьмем строку $abacababacabc$.

Словарь	Осталось обработать	Найденный префикс	Код	Примечание
$\varnothing$	$abacababacabc$	—	$\langle 0, a\rangle$	В словаре ничего не нашлось, вместо номера в словаре указываем 0
a	$bacababacabc$	—	$\langle 0, b\rangle$	
a, b	$acababacabc$	a	$\langle 1, c\rangle$	Нашелся префикс a (слова в словаре нумеруются с 1), добавляем ac
a, b, ac	$ababacabc$	a	$\langle 1, b\rangle$	
a, b, ac, ab	$abacabc$	ab	$\langle 4, a\rangle$	
a, b, ac, ab, aba	$cabc$	—	$\langle 0, c\rangle$	
a, b, ac, ab, aba, c	abc	ab	$\langle 4, c\rangle$	

Результатом кодирования является список пар: $[\langle 0, a\rangle, \langle 0, b\rangle, \langle 1, c\rangle, \langle 1, b\rangle, \langle 4, a\rangle, \langle 0, c\rangle, \langle 4, c\rangle]$

LZ78

```
struct Node:
    int pos   // номер слова в словаре
    char next
```

```
list<Node> encodeLZ78(string s):
    string buffer = ""                              // текущий префикс
    map<string, int> dict = {}                      // словарь
    list<Node> ans = []                             // ответ
    for i = 0 .. s.length - 1:
        if dict.hasKey(buffer + s[i]):              // можем ли мы увеличить
префикс
            buffer += s[i]
        else:
            ans.push({dict[buffer], s[i]})          // добавляем пару в ответ
            dict[buffer + s[i]] = dict.length + 1   // добавляем слово в
словарь
            buffer = ""                             // сбрасываем буфер
    if not (buffer is empty): // если буффер не пуст - этот код уже был,
нужно его добавить в конец словаря
        last_ch = buffer.peek() // берем последний символ буффера, как
"новый" символ
        buffer.pop() // удаляем последний символ из буфера
        ans.push({dict[buffer], last_ch}) // добавляем пару в ответ
    return ans
```

LZ78

Декодирование

Декодирование происходит аналогично кодированию, на основе декодируемой информации строим словарь и берем из него значения.

```
string decodeLZ78(list<Node> encoded):
    list<string> dict = [""]                        // словарь, слово с номером 0 — пустая строка
    string ans = ""                                 // ответ
    for node in encoded:
        word = dict[node.pos] + node.next           // составляем слово из уже известного из словаря и новой буквы
        ans += word                                 // приписываем к ответу
        dict.push(word)                             // добавляем в словарь
    return ans
```

Коды Грея

Двоичным кодом Грея порядка **n** называется последовательность всех **n**-битных кодов, в которой любые два соседних кода различаются ровно в одном разряде

Пример кодов Грея порядка 2:

- 00
- 01
- 11
- 10

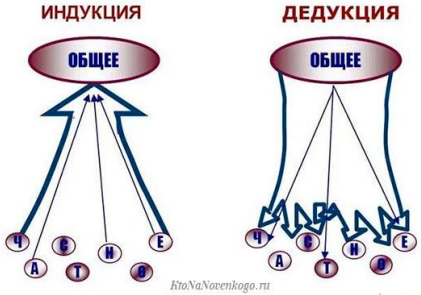
Заметим, что такая последовательность не единственная

Зачем?

Рассмотрим следующую задачу: «Дано множество из **n** элементов. Требуется вывести все его подмножества в таком порядке, что каждое следующее подмножество получается из предыдущего удалением или добавлением ровно одного элемента(т.е. в порядке минимального изменения).»

Будем считать, что элементы нашего множества пронумерованы от **1** до **n**. Тогда любому набору элементов множества можно поставить в соответствие **n**-битный двоичный код: если **i**-ый элемент множества входит в набор, то **i**-ый бит кода равен единице, иначе нулю. Таким образом, перебор всех подмножеств сводится к перебору двоичных кодов порядка **n**, а перебор подмножеств в порядке минимального изменения — к перебору двоичных кодов Грея.

Доказательство существования



Существование кодов Грея легко доказывается по индукции.

База - n = 0 - пустая последовательность.

Переход. Пусть G_n - последовательность кодов Грея порядка n. G^R_n - последовательность G_n записанная в обратном порядке. Тогда последовательность G_{n+1} получается приписыванием нуля слева к каждому коду в последовательности G_n и единицы слева к каждому коду G^R_n и последующим их объединением, т.е. $G_{n+1} = 0G_n \cup 1G^R_n$.

Нетрудно проверить, что свойство кодов Грея в последовательности G_{n+1}, построенную таким образом, не нарушается.

Принцип построения кода Грея

Понятное объяснение:

Начнем с двух чисел, закодируем их нулем и единицей:

Ноль 0

Один 1

Заметим, что они отличаются одним битом. Проблема в том, что нам нужно больше чем два числа. Зеркально отразим коды чисел:

Ноль 0

Один 1

— — — — (ось симметрии)

1

0

Кодов стало больше, и соседи отличаются друг от друга не более чем одним битом. Но есть повторы. Чтобы не было повторов, допишем спереди к первой половине кодов 0, а ко второй 1

Ноль 00

Один 01

— — — —

Два 11

Три 10

В общем виде

В общем виде

Должно быть уже понятно, но на всякий случай:

0	00000	
1	00001	
2	00011	зеркально отражается младший разряд
3	00010	
...	...	
2^n-2	00...000 | gray(2^n-2)	
2^n-1	00...000 | gray(2^n-1)	
2^n	00...001 | gray(2^n-1)	очередное отражение
2^n+1	00...001 | gray(2^n-2)	
2^n+2	00...001 | gray(2^n-3)	
...	...	
2^{n+1}-2	00...001 | gray(1)	
2^{n+1}-1	00...001 | gray(0)	обратите внимание на то, чему равен этот код
2^{n+1}	00...011 | gray(0)	
...	...	

Примеры: число $9_{10} = 1001_2$.

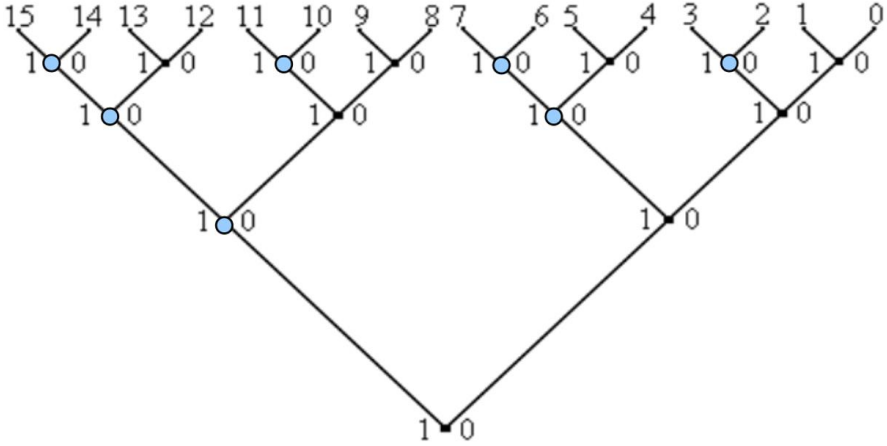
$$\begin{array}{rl} 1001 & \text{— исходный код} \\ \oplus\ \underline{100} & \text{— сдвинутый код без младшего разряда} \\ 1101 & \text{— код Грея.} \end{array}$$

число $15_{10} = 1111_2$

$$\begin{array}{rl} 1111 & \text{— двоичный код} \\ \oplus\ \underline{111} & \text{— сдвинутый код без младшего разряда} \\ 1000 & \text{— код Грея.} \end{array}$$

Коды Грея через дерево

Для получения по этой схеме кода Грея нужно в каждой второй справа в своем ряду «вилке» (на рисунке эти узлы отмечены знаком ○) поменять местами 1 и 0. Теперь при прохождении по измененному графу от корня до концевых узлов мы сможем определять код Грея для выбранного числа.



Декодирование кодов Грея

Для получения числа в двоичном коде из кода Грея нужно пошагово сдвигать число вправо на бит и Xor его с числом полученным на предыдущем шаге, пока не останется нуля.

$$
\begin{array}{rr}
 & 10110011 \text{ – число в коде Грея} \\
\oplus & 1011001 \\
\oplus & 101100 \\
\oplus & 10110 \\
\oplus & 1011 \\
\oplus & 101 \\
\oplus & 10 \\
\oplus & \underline{\qquad 1} \\
 & 11011101 \text{ – число в двоичном коде.}
\end{array}
$$

Построение кода Грея из натурального двоичного кода

Для перехода от натурального двоичного кода к коду Грея существуют правила:

- если в предыдущем разряде двоичного кода стоит 0, то в данном разряде цифра сохраняется
- иначе (стоит 1), то в данном разряде цифра меняется на противоположную.

A_{10}	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*
A_2	0000	0001	001<u>0</u>	001<u>1</u>	01<u>0</u>0	01<u>0</u>1	01<u>10</u>	01<u>11</u>
Код Грея	0000	0001	0011	0010	0110	0111	0101	0100
A_{10}	*8*	*9*	*10*	*11*	*12*	*13*	*14*	*15*
A_2	1<u>0</u>00	1<u>0</u>01	1<u>0</u>1<u>0</u>	1<u>0</u>1<u>1</u>	1<u>10</u>0	1<u>10</u>1	1<u>110</u>	1<u>111</u>
Код Грея	1100	1101	1111	1110	1010	1011	1001	1000

Псевдокод

- GrayCode - двумерный массив boolean, в котором GrayCode[a][b] - b-ый бит в a-ом коде Грея.
- p - счетчик количества уже имеющихся кодов.
- t - Показывает количество в (a-1)-м коде Грея.

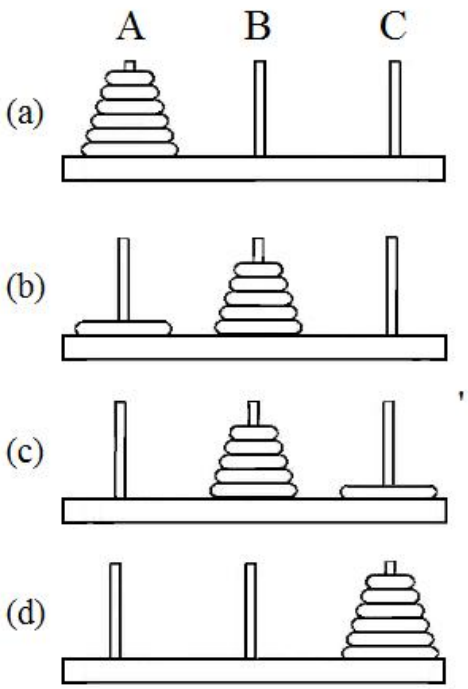
```
buildCode(n):
   GrayCode[1, n] = false
   GrayCode[2, n] = true                    // Построение кода длины 1
   p = 2
   for i = 2 to n
      t = p
      p = p * 2
      for k = (p / 2 + 1) to p
         GrayCode[k] = GrayCode[t]          // Отражение имеющихся кодов
         GrayCode[t, n + 1 - i] = false
         GrayCode[k, n + 1 - i] = true      // Добавление 0 и 1 в начало
         t--
   return GrayCode
```

Какие задачи решаются кодами Грея

- Коды Грея используются для кодирования номера дорожек в жёстких дисках.
- Коды Грея широко применяются в теории генетических алгоритмов для кодирования генетических признаков, представленных целыми числами.
- Коды Грея используются в Картах Карно (при передаче в карту переменные сортируются в код Грея).
- Алгоритм модуляции 2B1Q (англ. *2 Binary 1 Quandary*)

Задача о Ханойских башнях

Даны три стержня, на один из которых нанизаны восемь колец, причем кольца отличаются размером и лежат меньшее на большем. Задача состоит в том, чтобы перенести пирамиду из восьми колец за наименьшее число ходов на другой стержень. За один раз разрешается переносить только одно кольцо, причём нельзя класть большее кольцо на меньшее.



Решение

Пусть n — количество дисков. Начнём с кода Грея длины n, состоящего из одних нулей (т.е. $G(0)$), и будем двигаться по кодам Грея (от $G(i)$ переходить к $G(i+1)$).

Поставим в соответствие каждому i-ому биту текущего кода Грея i-ый диск (причём самому младшему биту соответствует наименьший по размеру диск, а самому старшему биту — наибольший). Поскольку на каждом шаге изменяется ровно один бит, то мы можем понимать изменение бита i как перемещение i-го диска. То есть, будем понимать переход от последовательности 101 к 100 как перемещение 0-го диска на свободное место, а от 010 к 110 — как перемещение 2-го диска на свободное место.

Заметим, что для всех дисков, кроме наименьшего, на каждом шаге имеется ровно один вариант хода (за исключением стартовой и финальной позиций). Для самого маленького диска всегда есть две свободные позиции, потому что он самый маленький, его можно положить сверху на любой диск. Если диск не самый маленький, то для него может быть не более одной свободной позиции. Допустим, что для него свободные две позиции. Это означает, что на двух других стержнях расположены диски размером больше, чем рассматриваемый. А так как рассматриваемый диск не самый маленький, то где-то расположен наименьший. Либо он расположен на рассматриваемом диске, тогда мы не можем переместить рассматриваемый, либо на каком-то другом, но тогда у нашего диска остаётся не более одной свободной позиции. Для наименьшего диска всегда имеется два варианта хода, однако имеется стратегия выбора хода, всегда приводящая к ответу: если n нечётно, то последовательность перемещений наименьшего диска имеет вид $r_1 \to r_3 \to r_2 \to r_1 \to r_3 \to r_2 \to \dots$.(где r_1 — стартовый стержень, r_3 — финальный стержень, r_2 — оставшийся стержень), а если n чётно, то $r_1 \to r_2 \to r_3 \to r_1 \to r_2 \to r_3 \to \dots$.

Шифрование

- **Шифр** - система преобразования текста для обеспечения секретности передаваемой информации.
- Процесс засекречивания сообщения с помощью шифра называется **шифрованием**.
- Наука о создании и использовании шифров называется **криптографией**. Наука о методах получения исходного значения зашифрованной информации называется **криптоанализ**.
- Исходное сообщение называется в криптографии открытым **текстом**, или **клером**. Засекреченное (зашифрованное) сообщение называется шифротекстом, или шифрограммой, или **криптограммой**.

Ключи и типы шифров

Важным параметром любого шифра является **ключ** — параметр криптографического алгоритма, обеспечивающий выбор одного преобразования из совокупности преобразований, возможных для этого алгоритма.

Симметричный шифр использует один ключ для шифрования и расшифрования.

- Простая перестановка
- Одиночная перестановка по ключу

Асимметричный шифр использует два различных ключа.

- PGP
- TLS

Блочный шифр шифрует сразу целый блок текста

- SP-сеть
- Сеть Фейстеля

Поточный шифр шифрует информацию по мере поступления

Алгоритм замены

Один из способов шифрования – **простая замена**, при которой каждая буква открытого текста заменяется на какую-то букву алфавита (возможно, на ту же самую). Для этого отправитель сообщения должен знать, на какую букву в шифротексте следует заменить каждую букву открытого текста.

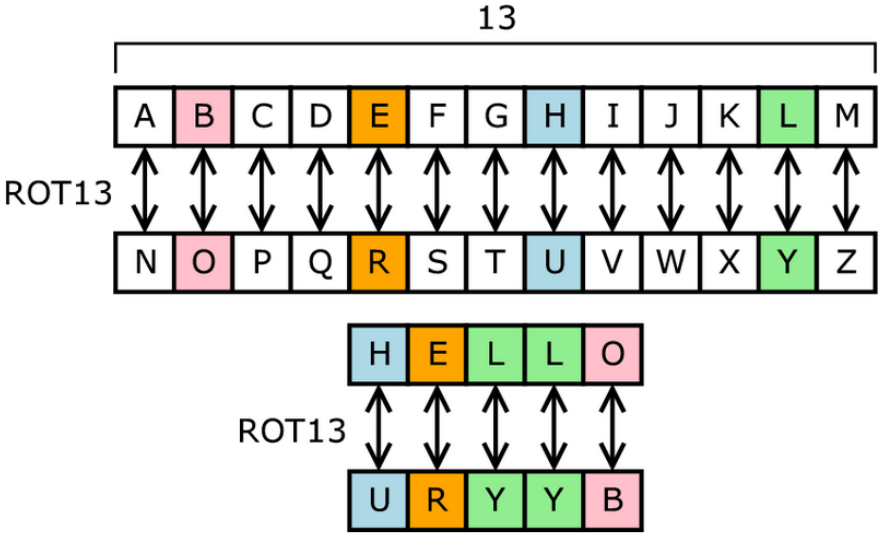
Например:

Открытый Алфавит	А	Л	М	Р	У	Ы
Шифровальный алфавит	А	Б	В	Г	Д	Е

Открытый текст: МАМА МЫЛА РАМУ
Зашифрованный текст: ВАВА ВЕБА ГАВД

Алгоритм замены

```
for(a=0;a<i;a++)// рандомное заполнение ключа
        key[a]=char((rand()%255));
for(a=0;a<i;a++)// шифрование
        text[a]+=key[a];
for(a=0;a<i;a++)// дешифрование
        text[a]-=key[a];
```



Принцип работы программы ROT13 для скрытия информации

Алгоритм перестановки

В криптографическом **алгоритме перестановки** все **буквы** открытого текста остаются без изменений, но **переставляются** согласно заранее оговоренному правилу. Здесь также удобно использовать ключ, управляющий процедурой шифрования.

	4	1	3	2
3	П	Р	И	Л
1	Е	Т	А	Ю
4	В	О	С	Ь
2	М	О	Г	О

Исходная таблица

	1	2	3	4
3	Р	Л	И	П
1	Т	Ю	А	Е
4	О	Ь	С	В
2	О	О	Г	М

Перестановка столбцов

	1	2	3	4
1	Т	Ю	А	Е
2	О	О	Г	М
3	Р	Л	И	П
4	О	Ь	С	В

Перестановка строк

Пример использования алгоритма перестановки

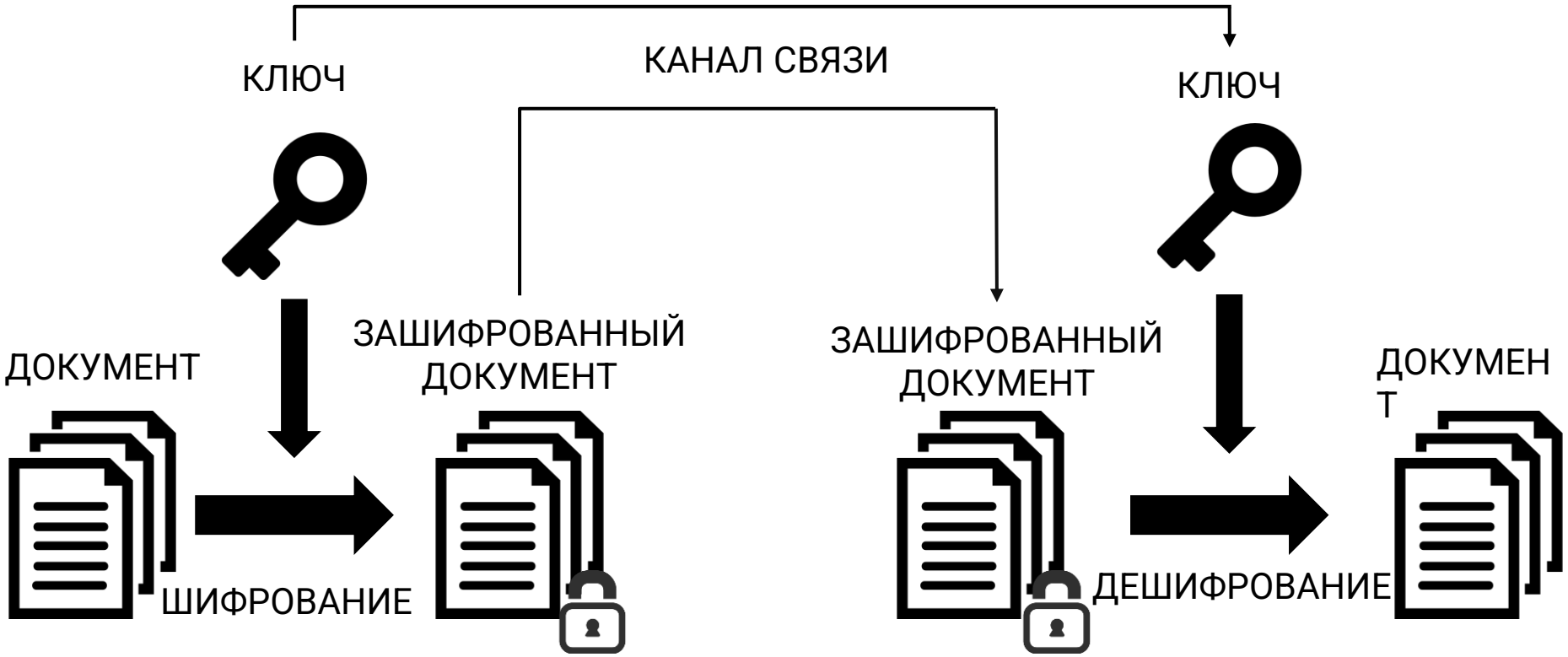
Алгоритм перестановки

- В цикле (строки 1-10) проходимся по всем блокам исходного сообщения длиной в ключ.
- На каждой итерации этого цикла создается символьный массив transposition (строка 3), в который будем заносить зашифрованный текст (переставленные символы) — символы в новом порядке исходя из ключа.
- Затем во внутреннем цикле (строки 5-6) перебираем все символы блока и происходит непосредственно шифрование (перестановка) по ключу (строка 6).
- После в цикле (строки 8-9) посимвольно прибавляем зашифрованный блок к концу строковой переменной с результатом (строка 9).

```
1   for (int i = 0; i < input.Length; i += key.Length)
2   {
3       char[] transposition = new char[key.Length];
4
5       for (int j = 0; j < key.Length; j++)
6           transposition[key[j] - 1] = input[i + j];
7
8       for (int j = 0; j < key.Length; j++)
9           result += transposition[j];
1   }
0
```

Возьмем в качестве ключа для сообщения МАМА МЫЛА РАМУ слово БАЙТ. Пронумеруем буквы ключевого слова в порядке их следования слева направо в русском алфавите. Далее под полученной числовой последовательностью в строках, равных по длине ключевому слову, запишем открытый текст.

Буквы ключевого слова	Б	А	Й	Т
Порядковые номера букв в алфавите	2	1	3	4
Открытый текст	М	А	М	А
	М	Ы	Л	А
	Р	А	М	У

Открытый текст: МАМА МЫЛА РАМУ
Зашифрованный текст: АЫАММРМЛМААУ

Передача зашифрованной информации